

ARTICTON New Device Setup Guide

GitHub pull, local environment recreation, Firebase setup, and local testing

Use this when moving the project to a new laptop or PC and the local .env files are missing.

Project ID	articton-57fd8
Frontend	React + Vite
Backend	Firebase Cloud Functions
Local AI behavior	Uses the improved fallback tutor unless Gemini billing/key is available.
Important	Do not commit .env, .env.local, API keys, app passwords, or Firebase secret values.

1. Save the Current Device Changes

Run these from the current project folder before moving to the new device.

1. Check what changed.
2. Commit the updates.
3. Push the branch to GitHub.

```
git status
```

```
git add .
```

```
git commit -m "Update tutor fallback and voice over"
```

```
git push origin main
```

2. New Device Setup

- Install Git, Node.js LTS, and VS Code.
- Use Firebase CLI through npx so it does not need a global install.
- Sign in to the Google account that has Firebase project access.

```
git clone <your-github-repo-url>
```

```
cd ARTICTION
```

If the project already exists on the new device, use this instead:

```
git pull origin main
```

3. Install Dependencies

```
npm install
```

```
cd functions
```

```
npm install
```

```
cd ..
```

4. Recreate Frontend Env Files

The real env files are intentionally ignored by Git, so create fresh local copies.

```
copy .env.example .env
```

```
VITE_FIREBASE_API_KEY=<your Firebase web api key>
```

```
VITE_DEV_BYPASS_LOGIN=false
```

Create .env.local for local function routing:

```
VITE_USE_FUNCTIONS_EMULATOR=false
```

Use false when npm run dev should call the deployed Firebase Functions. Use true only when the Functions emulator is running.

5. Recreate Functions Env

This file is only for local Functions emulator testing. Deployed Functions should use Firebase secrets.

```
copy functions\.env.example functions\.env
```

```
GMAIL_USER=<sender Gmail address>
```

```
GMAIL_APP_PASSWORD=<Gmail app password>
```

```
OTP_HASH_SECRET=<random long secret for OTP hashing>
```

6. Connect Firebase CLI

```
npx -y firebase-tools@latest login
```

```
npx -y firebase-tools@latest use
```

If articton-57fd8 is not selected, run:

```
npx -y firebase-tools@latest use articton-57fd8
```

7. Run Local Tests

Start the frontend and test login, OTP, module scenes, fallback tutor, and voice-over.

```
npm run dev
```

Run a production build check before deployment:

```
npm run build
```

8. Optional Functions Emulator

Only use this if you want local function execution instead of deployed functions.

```
npx -y firebase-tools@latest emulators:start --only functions
```

```
VITE_USE_FUNCTIONS_EMULATOR=true
```

9. Gemini and AI Notes

Without Gemini billing	The app can still use the improved fallback tutor and browser voice-over for testing.
With Gemini later	Set the Gemini key/secret, enable the Gemini API, and turn on the tutor flag when billing is ready.
Avoid charges now	Keep Gemini disabled and rely on fallback tutor responses during local testing.
Deleted server folder	The old server folder is no longer needed if Firebase Functions handles backend work.

Quick Troubleshooting

Login says invalid account	Confirm the account exists in the app's expected Firebase collection/Auth flow, and test OTP using the latest email only.
OTP required error	Make sure the email value is preserved when moving from the email step to the OTP step.
AI repeats fallback text	Expected while Gemini is disabled or rejected. Test the fallback tutor and voice-over locally.
Firebase permission issue	Use the account with project access, billing access, and payment profile permissions as needed.
Do not deploy secrets	Never push .env files. Use Firebase secrets for production-only sensitive values.